Lecture 7 Compendium: Computer Imaging & Vision in Agriculture

Module 2: Technologies of IoT in Crop Production

Dr. Kadeghe Fue
PhD Agroecology

Compendium for Advanced Study

Contents

1 Introduction: From Photons to Phenotypes **2**

2 Traditional Image Processing (Feature Engineering) **2**
2.1 Color Spaces 2
2.2 Vegetation Segmentation 2

3 Deep Learning: Convolutional Neural Networks (CNNs) **2**
3.1 The Convolution Operation 2
3.2 Key Architectures 3

4 Data Engineering: The "Small Data" Problem **3**
4.1 Data Augmentation 3
4.2 Transfer Learning 3

5 Python Lab: Building a Plant Disease Classifier **3**
5.1 Implementation 4

6 Deployment: Edge AI Optimization **5**
6.1 Quantization 5
6.2 Format Conversion 5

7 Review Questions & Lab Assignment **6**

1 Introduction: From Photons to Phenotypes

Computer Vision (CV) in agriculture is the process of extracting meaningful information from digital images. It transforms a passive camera into an active diagnostic sensor.

For a PhD student, it is crucial to distinguish between:

- **Image Processing:** Manipulating the image (e.g., enhancing contrast, removing noise). The output is another image.
- **Computer Vision:** Analyzing the image to make a decision (e.g., "Is this a weed?"). The output is information.

2 Traditional Image Processing (Feature Engineering)

Before Deep Learning (2012), engineers manually crafted features to detect objects. While "old school," these methods are computationally cheap and explainable, making them relevant for low-power edge devices.

2.1 Color Spaces

The RGB color space is sensitive to lighting conditions (shadows/glare). In agriculture, we often convert to spaces that separate **Chrominance** (Color) from **Luminance** (Brightness).

- **HSV (Hue, Saturation, Value):** Hue (0-360) represents the "Color". Green plants typically fall in the Hue range of 30° to 90°, regardless of how bright the sun is.
- **Lab (CIELAB):** Perceptually uniform. L is Lightness, a is Green-Red, b is Blue-Yellow.

2.2 Vegetation Segmentation

To separate crop from soil without AI, we use the **Excess Green Index (ExG)**.

$$ExG = 2G - R - B \tag{1}$$

Since plants reflect more Green than Red or Blue, pixel regions with high ExG are likely vegetation. This is a fast, robust "Vegetation Mask."

3 Deep Learning: Convolutional Neural Networks (CNNs)

Modern Computer Vision is dominated by CNNs. Instead of manual feature engineering (ExG), the network *learns* the optimal features from the data.

3.1 The Convolution Operation

The core of a CNN is the **Convolution**. A small matrix (Kernel/Filter) slides over the image, performing element-wise multiplication and summation.

For an Input Image I and Kernel K:

$$(I * K)(i, j) = \sum_m \sum_n I(i + m, j + n)K(m, n) \tag{2}$$

What does it learn?

- **Early Layers:** Detect simple edges (vertical, horizontal) and colors.
- **Middle Layers:** Detect textures and simple shapes (circles, corners).
- **Deep Layers:** Detect complex objects (leaves, insect wings, lesions).

3.2 Key Architectures

1. **ResNet (Residual Networks):** Uses "skip connections" to allow training of very deep networks (50+ layers) without the Vanishing Gradient problem. The standard backbone for classification.
2. **YOLO (You Only Look Once):** A single-stage detector. It divides the image into a grid and predicts bounding boxes and class probabilities simultaneously. Extremely fast (>45 FPS), making it ideal for **Real-Time Weed Spraying**.
3. **Mask R-CNN:** Two-stage detector. It provides instance segmentation (pixel-perfect masks). Slower, but critical for robotic harvesting (knowing exactly where the fruit edge is).

PhD Research Context

The Trade-off: YOLO is fast but less accurate on small objects (e.g., tiny insects). Faster R-CNN is accurate but too slow for a drone flying at 5 m/s. Your research may involve optimizing this trade-off for a specific Tanzanian pest.

4 Data Engineering: The "Small Data" Problem

Agriculture lacks massive, labeled datasets like ImageNet (14 million images). We often have only a few hundred images of a specific disease (e.g., Cassava Mosaic Disease).

4.1 Data Augmentation

Artificially expanding the dataset by applying transformations that do not change the label.

- **Geometric:** Rotation, Flip, Zoom, Shear. (Makes the model invariant to orientation).
- **Photometric:** Brightness, Contrast, Hue Jitter. (Makes the model invariant to lighting conditions).
- **CutOut / MixUp:** Advanced techniques to prevent overfitting.

4.2 Transfer Learning

The most important technique for agricultural AI.

1. Take a model pre-trained on ImageNet (knows how to detect edges/shapes).
2. **Freeze** the early convolutional layers (the "Feature Extractor").
3. **Replace** the final Fully Connected layers (the "Classifier") with new layers for your specific classes (e.g., Healthy vs. Diseased).
4. **Fine-tune** with a small learning rate.

5 Python Lab: Building a Plant Disease Classifier

We will implement a complete pipeline using **PyTorch**. We will define a custom dataset, apply augmentations, and fine-tune a ResNet18 model.

5.1 Implementation

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, models, transforms
from torch.utils.data import DataLoader

# --- 1. DATA PREPARATION ---
# Define transforms (Augmentation for train, Normalization for val)
data_transforms = {
    'train': transforms.Compose([
        transforms.RandomResizedCrop(224),
        transforms.RandomHorizontalFlip(),
        transforms.ColorJitter(brightness=0.2, contrast=0.2),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ]),
    'val': transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ]),
}

# Assume dataset is structured as: data/train/class_name/image.jpg
data_dir = 'data/plant_village_subset'
image_datasets = {x: datasets.ImageFolder(os.path.join(data_dir, x),
                                          data_transforms[x])
                  for x in ['train', 'val']}
dataloaders = {x: DataLoader(image_datasets[x], batch_size=32, shuffle=True)
               for x in ['train', 'val']}

# --- 2. MODEL SETUP (TRANSFER LEARNING) ---
# Load pre-trained ResNet18
model = models.resnet18(pretrained=True)

# Freeze all layers
for param in model.parameters():
    param.requires_grad = False

# Replace the final fully connected layer (fc)
# num_ftrs = input features of the last layer
num_ftrs = model.fc.in_features
class_names = image_datasets['train'].classes
# New layer (unfrozen by default)
model.fc = nn.Linear(num_ftrs, len(class_names))

device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
model = model.to(device)

# --- 3. TRAINING LOOP ---
criterion = nn.CrossEntropyLoss()
# Only optimize parameters of the final layer
optimizer = optim.SGD(model.fc.parameters(), lr=0.001, momentum=0.9)

def train_model(model, criterion, optimizer, num_epochs=5):
    for epoch in range(num_epochs):
        print(f'Epoch {epoch}/{num_epochs - 1}')

        for phase in ['train', 'val']:
            if phase == 'train':
```

```
                model.train()
            else:
                model.eval()

            running_loss = 0.0
            running_corrects = 0

            for inputs, labels in dataloaders[phase]:
                inputs = inputs.to(device)
                labels = labels.to(device)

                optimizer.zero_grad()

                with torch.set_grad_enabled(phase == 'train'):
                    outputs = model(inputs)
                    _, preds = torch.max(outputs, 1)
                    loss = criterion(outputs, labels)

                    if phase == 'train':
                        loss.backward()
                        optimizer.step()

                running_loss += loss.item() * inputs.size(0)
                running_corrects += torch.sum(preds == labels.data)

            epoch_loss = running_loss / len(image_datasets[phase])
            epoch_acc = running_corrects.double() / len(image_datasets[phase])

            print(f'{phase} Loss: {epoch_loss:.4f} Acc: {epoch_acc:.4f}')

    return model

# Run training
# trained_model = train_model(model, criterion, optimizer, num_epochs=10)
```

Listing 1: PyTorch Training Loop for Plant Disease

6 Deployment: Edge AI Optimization

Training a model is only half the battle. A ResNet model might be 100MB in size. This is too large to update over a slow 2G connection in rural Tanzania, and too slow to run on a battery-powered microcontroller.

6.1 Quantization

Converting the model weights from 32-bit Floating Point (Float32) to 8-bit Integers (Int8).

- **Size Reduction:** 4x smaller (100MB $\rightarrow$ 25MB).
- **Speed:** Faster inference on CPUs/DSPs.
- **Accuracy Cost:** Usually negligible ($<$ 1

6.2 Format Conversion

- **ONNX (Open Neural Network Exchange):** Universal format for interoperability.
- **TensorFlow Lite (.tflite):** Optimized for Android/Embedded Linux.
- **TensorRT:** Optimized for NVIDIA GPUs (Jetson Nano).

7 Review Questions & Lab Assignment

1. **Architecture:** Why does a Convolutional Neural Network (CNN) generalize better to new unseen images compared to a Multi-Layer Perceptron (MLP) that treats the image as a flat vector?

2. **Metrics:** In a dataset where 95% of leaves are healthy and 5% are diseased, a model that predicts "Healthy" for everything achieves 95% accuracy. Why is this bad? What metric (Precision, Recall, F1-Score) should you use instead?

3. **Design:** You are building a camera trap to detect nocturnal pests. The background is static (the same field view). Is deep learning (YOLO) necessary, or could you use traditional Background Subtraction? Justify your choice based on power consumption.

References

1. Kamilaris, A., & Prenafeta-Boldú, F. X. (2018). *Deep learning in agriculture: A survey.* Computers and Electronics in Agriculture.

2. Redmon, J., et al. (2016). *You Only Look Once: Unified, Real-Time Object Detection.* CVPR.

3. PyTorch Documentation. `https://pytorch.org/tutorials/`